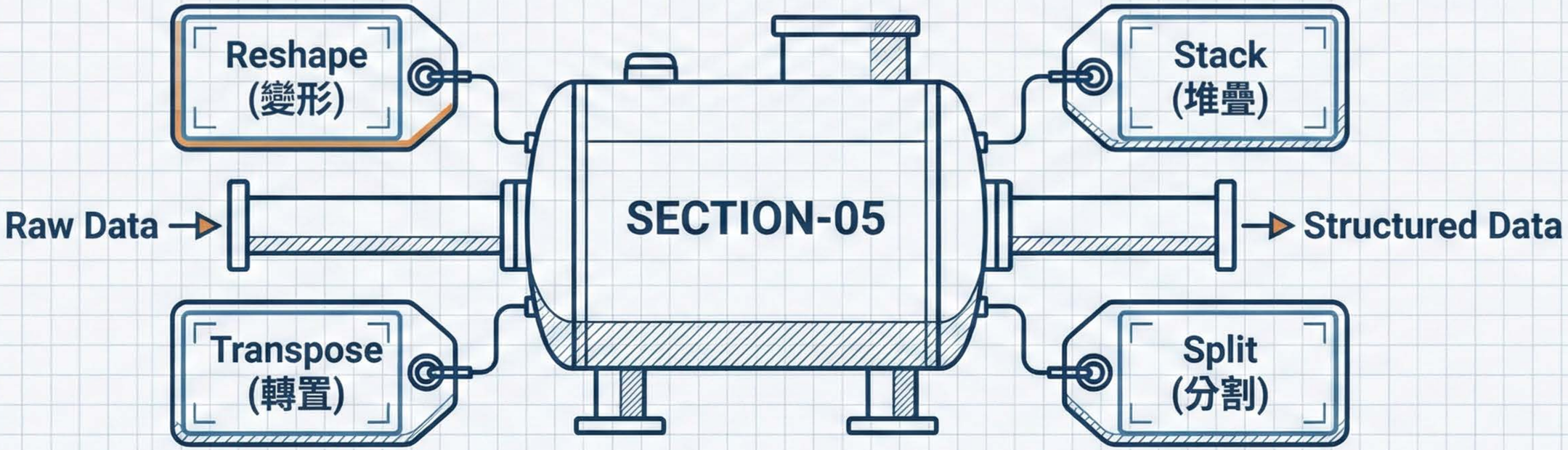
Section 5：陣列變形與組合

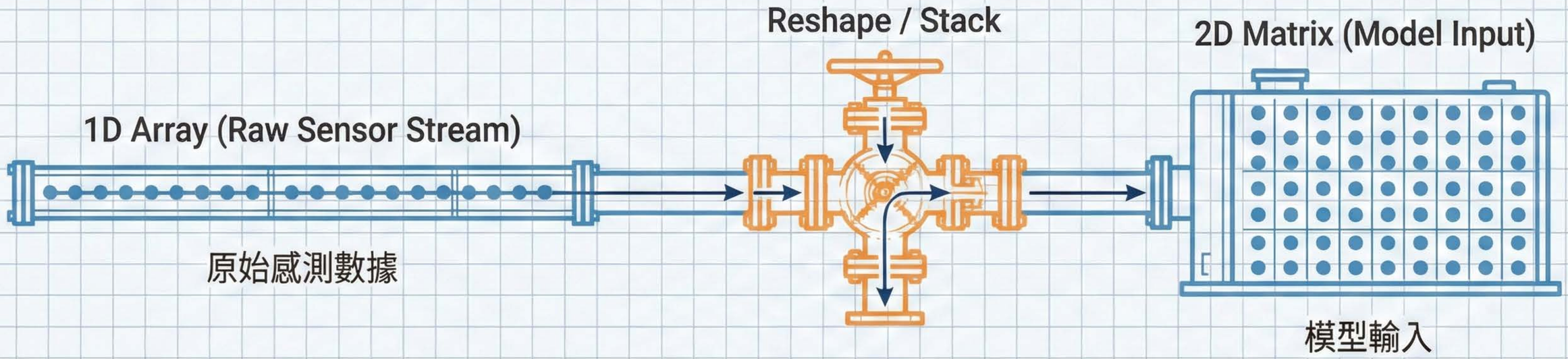
Array Reshaping and Combination

資料的單元操作：重塑、轉置、堆疊與分割



NotebookLM

為什麼需要改變資料形狀？
(Why Manipulate Shapes?)



數學模型需求
(Mathematical Requirements)

線性代數運算（如 $A \cdot B$）對維度有嚴格要求，資料必須適配矩陣規格。

特徵工程
(Feature Engineering)

將不同來源的數據（如溫度、壓力）整合為單一特徵矩陣。

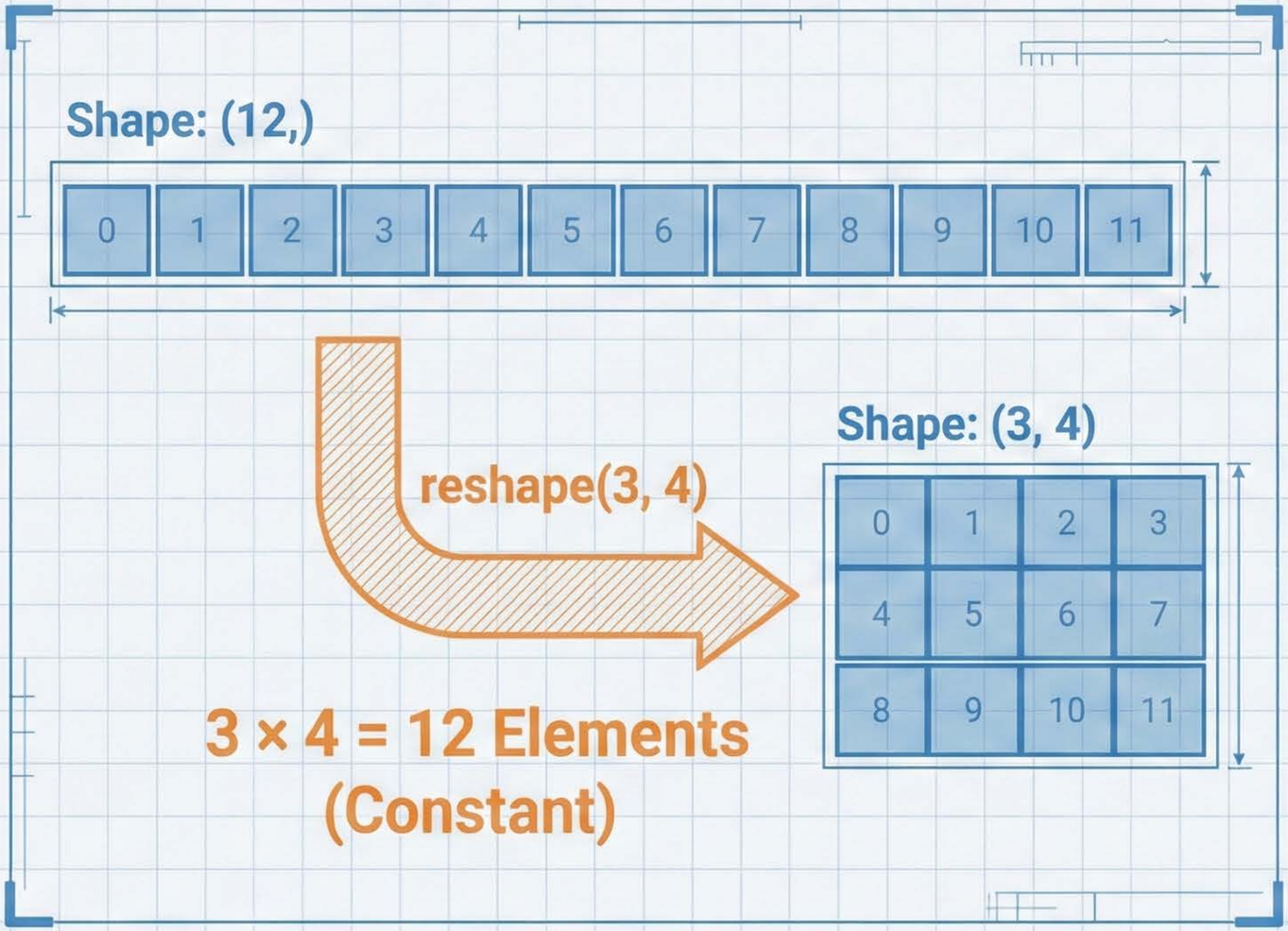
視覺化
(Visualization)

將 1D 序列數據轉換為 2D 網格以繪製熱圖（Heatmap）。

就像化工管線必須匹配管徑才能連接，數據也必須調整形狀才能進入運算單元。

5.1 改變陣列形狀 (Reshaping)

質量守恆定律：總元素數量必須保持不變



Good Code

```
import numpy as np

arr = np.arange(12)          # [0, 1, ..., 11]
new_arr = arr.reshape(3, 4) # 變形為 3 列 4 行
```

操作規則 (Rules)

- 不改變數據內容：僅改變數據的「視圖」(View) 與維度結構。
- 記憶體配置：NumPy 盡量不複製資料，而是共用記憶體 (Zero-copy)，效率極高。
- 限制：新形狀的乘積必須等於原陣列元素總數 (例如 12 個元素不能變形為 3x5)。

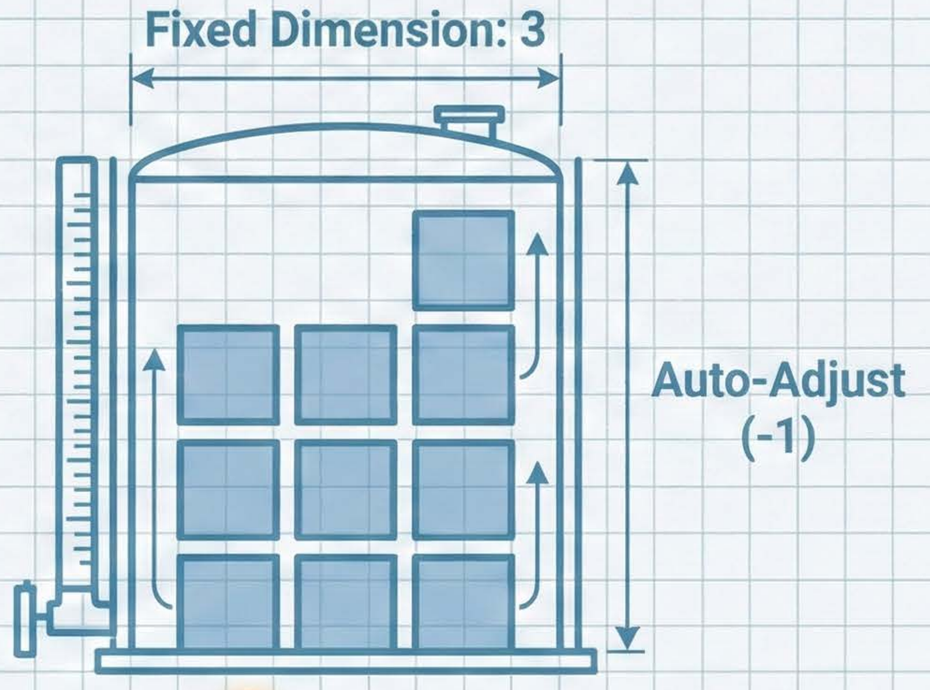
NotebookLM

5.1 進階變形技巧：自動維度推算

使用 `-1` 讓 NumPy 自動計算剩餘維度

Python Code

```
arr = np.arange(12)

# NumPy 自動計算行數：12 / 3 = 4
auto_arr = arr.reshape(3, -1)

# 常見應用：將一維陣列轉為「列向量」(Column Vector)
col_vector = arr.reshape(-1, 1)
```



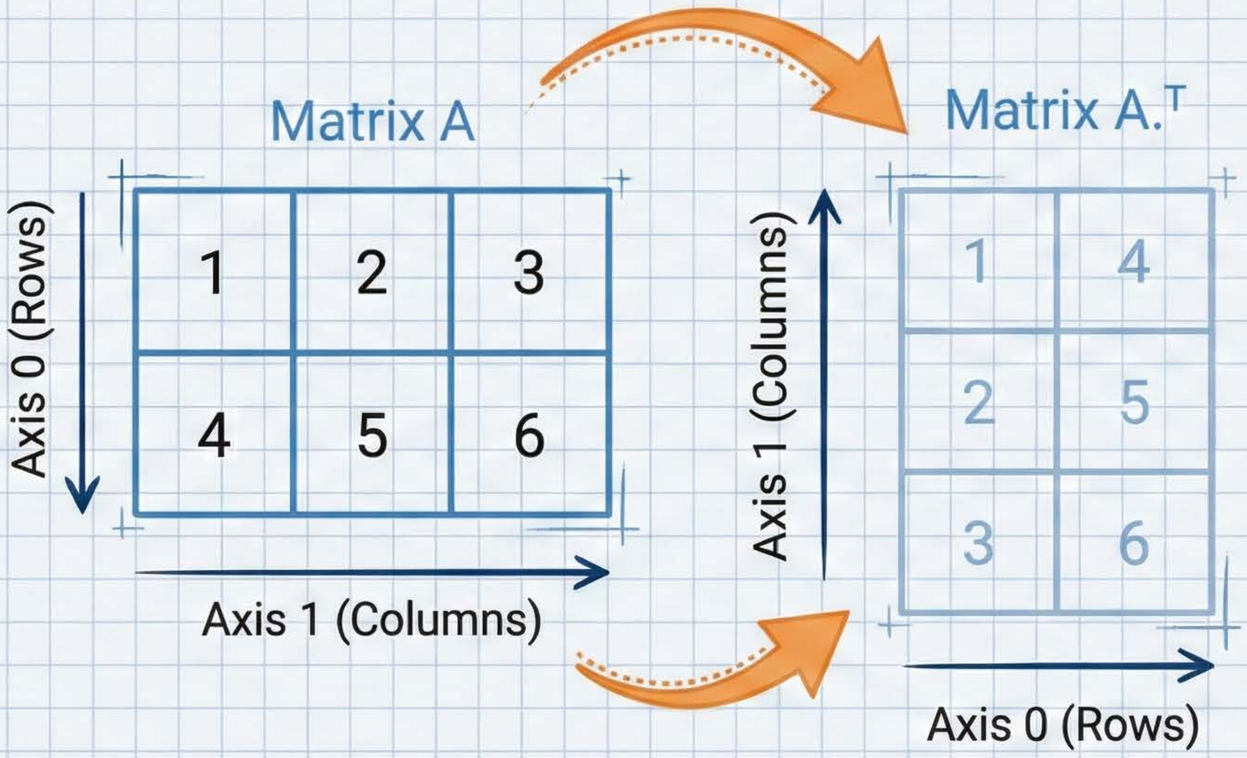
機器學習應用 (ML Application)

在 Scikit-learn 或 TensorFlow 中，特徵矩陣通常要求是 2D 結構 (n_samples, n_features)。當只有一個特徵時，必須使用 **.reshape(-1, 1)** 轉換格式。

NotebookLM

5.2 轉置 (Transpose)

改變觀察視角：軸的交換



Python Code

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])  # shape: (2, 3)

# 轉置操作
arr_T = arr.T          # shape: (3, 2)
# 或
arr_T = np.transpose(arr)
```

Technical Specs

- **定義**：將第 i 列第 j 行的元素移至第 j 列第 i 行。
- **應用場景**：矩陣乘法前調整維度、影像處理中交換長寬通道。

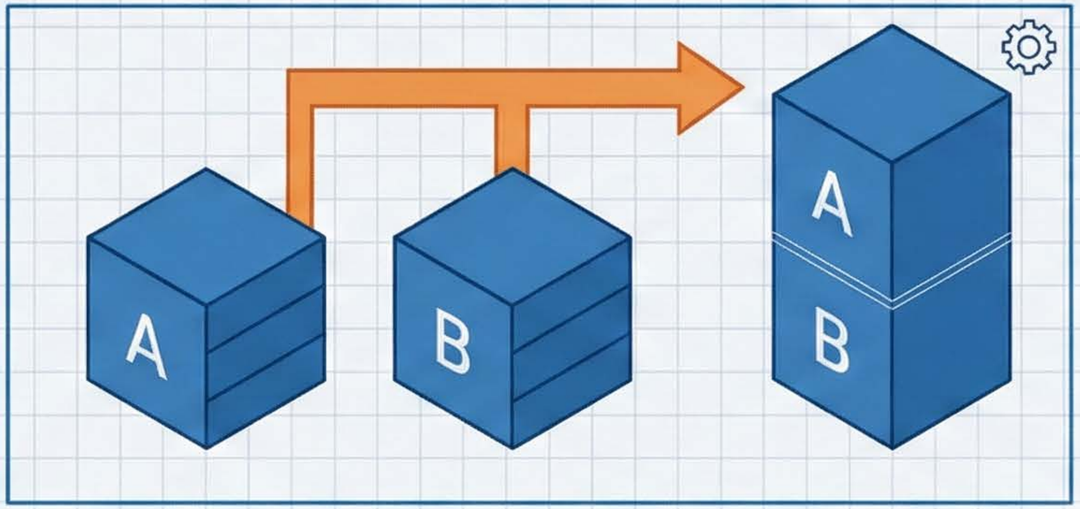
NotebookLM

5.3 陣列堆疊 (Stacking)

匯流操作：合併多個數據流

垂直堆疊 (Vertical Stack)

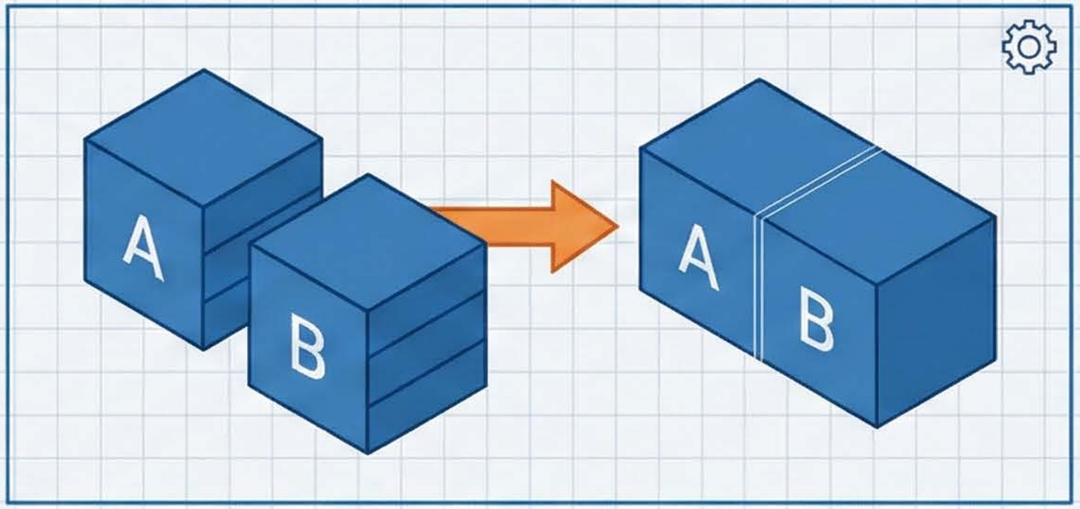
```
np.Vstack((a, b))
```



增加數據筆數 (Adding Samples)。就像在反應器記錄表中新增一天的數據。

水平堆疊 (Horizontal Stack)

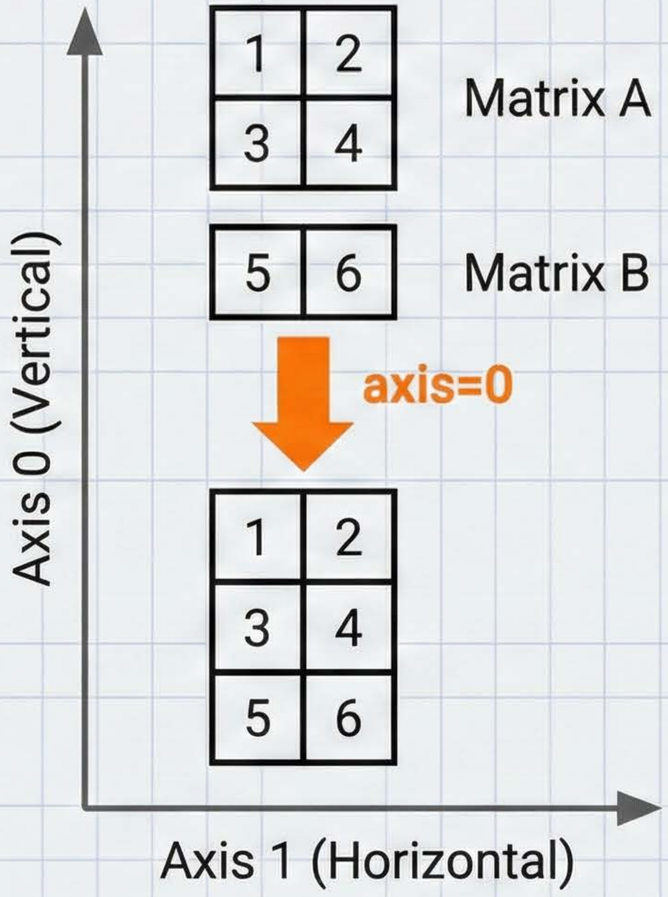
```
np.hstack((a, b))
```



增加特徵維度 (Adding Features)。就像將溫度數據與壓力數據並排合併。

5.3 指定軸向堆疊 (Concatenate)

`np.concatenate` 提供更靈活的維度控制



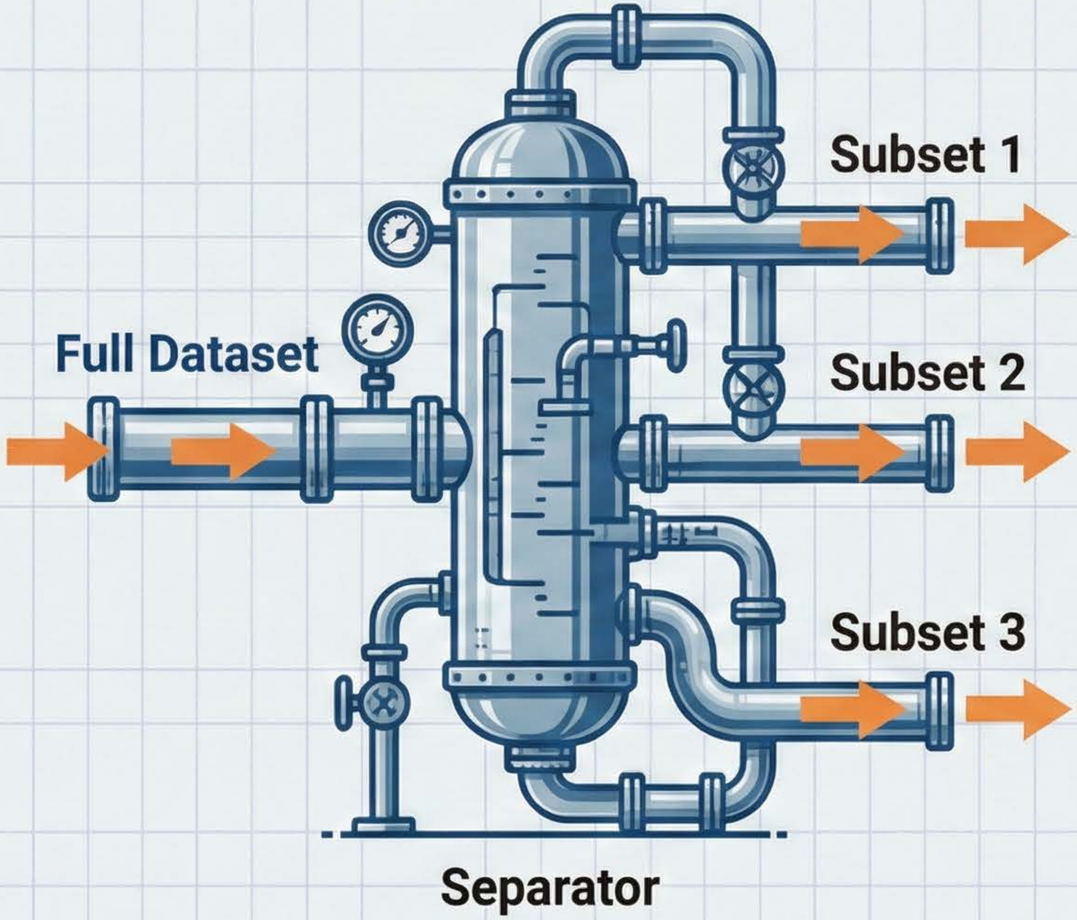
Python Code

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6]])

# 沿著 Axis 0 (垂直方向) 接合 -> 類似 vstack
result_0 = np.concatenate((A, B), axis=0)

# 沿著 Axis 1 (水平方向) 接合 -> 需注意維度匹配
# result_1 = np.concatenate((A, B.T), axis=1)
```

Dimension Check 除了堆量方向的軸 (Axis) 之外，其他維度的大小必須完全一致，否則會報錯 (ValueError)。

5.4 陣列分割 (Splitting)

分離程序：將數據流拆分為子集



Python Code

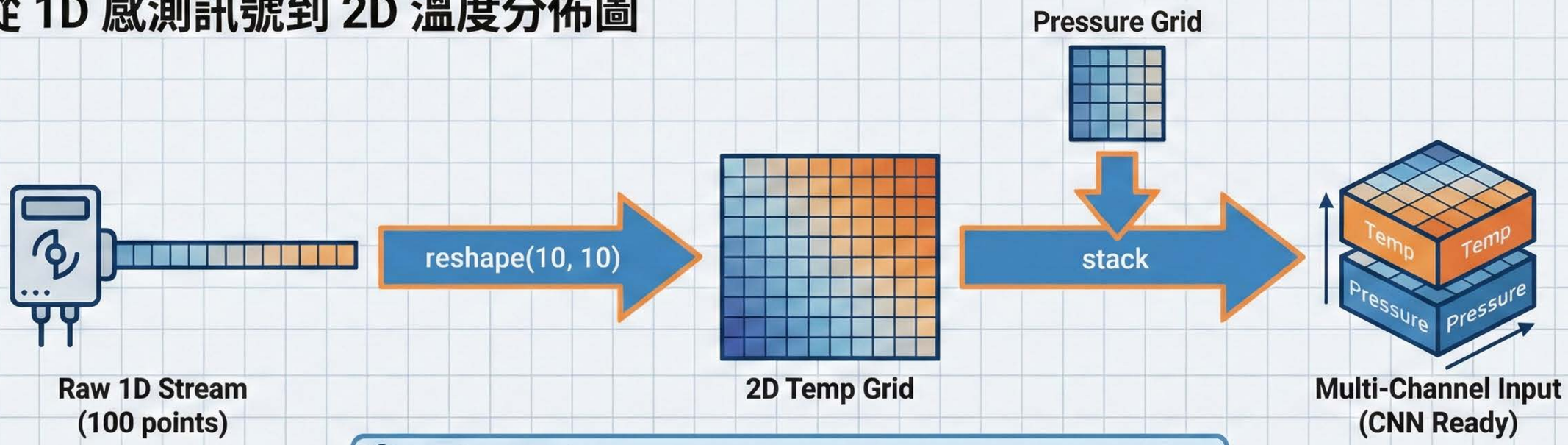
```python
arr = np.arange(9) # [0, 1, 2, ..., 8]

# 將陣列均分為 3 份
x, y, z = np.split(arr, 3)
# 結果: [0, 1, 2], [3, 4, 5], [6, 7, 8]

# 依索引位置切割（例如切出前 60% 訓練資料）
train, test = np.split(arr, [6])
```

應用場景 (Applications)

- **Machine Learning**：訓練集 (Training Set) 與測試集 (Test Set) 的分割。
- **Cross-Validation**：K-Fold 驗證時的數據切分。

實務應用案例：熱交換器數據前處理

從 1D 感測訊號到 2D 溫度分佈圖



Python Code

```
# 1. 重塑
temp_grid = raw_data.reshape(10, 10)
# 2. 堆疊 (Channels Last)
final_input = np.stack((temp_grid, press_grid), axis=-1)
```

⚠ 原始數據通常是序列化的，工程師必須利用 NumPy 恢復其空間或物理結構，才能進行正確的模擬分析。

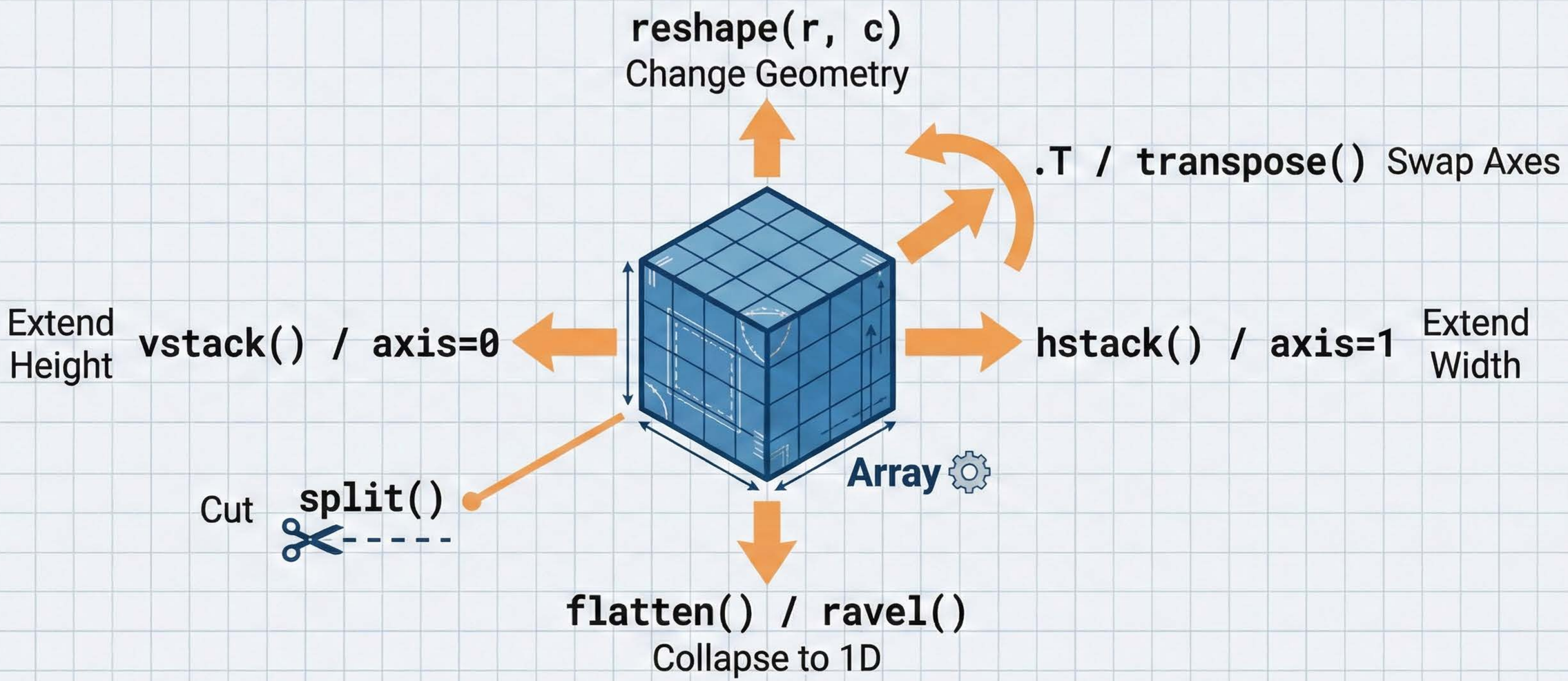
NotebookLM

單元操作總結與檢查表 (SOP Checklist)

「正確的數據形狀，是模型能夠收斂的第一步。」

NotebookLM

NumPy 形狀操作指令速查 (Quick Reference)



🔧 Note: flatten 產生副本 (Copy) ，ravel 通常回傳視圖 (View) 。

NotebookLM